



HA-D3QN: Embedding virtual private cloud in cloud data centers with heuristic assisted deep reinforcement learning

Meng Chen^a, Jiaxin Hou^a, Yongpan Sheng^a, Yingbo Wu^{a,*}, Sen Wang^a, Jianyuan Lu^b, Qilin Fan^a

^a School of Big Data and Software Engineering, Chongqing University, China

^b Alibaba Group, China

ARTICLE INFO

Article history:

Received 5 January 2023

Received in revised form 25 April 2023

Accepted 25 May 2023

Available online 1 June 2023

Keywords:

Virtual network embedding

Virtual private cloud

Deep reinforcement learning

Cloud data center

Switching capacity

Guaranteed bandwidth

ABSTRACT

The virtual network embedding problem is embedding virtual networks (VNs) in a substrate network so that revenue or accept ratio is maximized. Previous study usually assumes disclosed communication demand among the virtual nodes in a VN, mismatching real-world cloud computing scenarios. In this paper, we propose a new VN abstraction based on the widely used Virtual Private Cloud model, where internal communication demand is unknown to cloud providers. In contrast with the majority of existing research, we allow the co-location of the virtual nodes belonging to the same VN, and introduce the concept of switching capacity for practical resource reservation. We categorize the substrate resources in cloud data centers into additive and non-additive for the first time, and devise our algorithms accordingly. After formulating the problem, we propose a solution framework named HA-D3QN (Heuristic Assisted Dueling Double Deep Q Network). Essentially, HA-D3QN selects the best responses to different system states by combining the D3QN deep reinforcement learning structure and the candidate actions, which are generated by our proposed heuristic algorithms for addressing the exponentially large action space. Finally, we conduct extensive simulation experiments, the results of which verify the effectiveness of our approach.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

With the proliferation of virtualization technologies, an increasing number of computing tasks are moved from on-premise infrastructures to the cloud for better scalability, reduced cost, faster deployment, etc. The growing demand becomes a rising challenge for cloud providers: the effective and efficient provision and management of valuable resources in *Cloud Data Centers* (CDCs) are non-trivial, and have remained an active area of research. In this field, *Virtual Network Embedding Problem* (VNEP) emerges as one of the most challenging problems.

VNEP is usually described as embedding users' requests, represented by virtual networks (VNs), in a substrate network (SN) so that revenue or accept ratio is maximized [1,2]. While progress has been made in solving VNEP, existing study generally assumes the precise communication demand between any pair of virtual nodes in a VN is static and disclosed [3–6]. However, this assumption seems to be an over-simplification for CDCs because according to the common practice in real-world cloud

platforms, the communication demand among virtual machines can be rather dynamic and is usually kept private. Therefore, this assumption may limit even diminish the practicality of exiting solutions for VNEP in CDCs. Moreover, previous research usually prohibits the virtual nodes belonging to the same VN from being embedded in the same substrate node [3–6], also mismatching typical cloud computing scenarios because resource sharing on substrate nodes is enabled by virtualization technologies and such prohibition does not inherently exist. Sticking to this prohibition may lead to underestimating the embedding potentials of substrate nodes and consequently cause sub-optimal even poor results.

To address the above issues and solve VNEP in CDCs, in this paper, we propose a VN abstraction according to *Virtual Private Cloud* (VPC), the de facto model for organizing virtual networks in major cloud platforms (e.g., AWS [7], Google Cloud [8]). This VN abstraction is depicted by a full mesh of virtual nodes where each node is associated with attributes such as computing demand and guaranteed bandwidth. In sharp contrast to previous study, the links in our VNs are not assigned weights that represent static bandwidth demand, conforming to the common practice in cloud computing. Furthermore, we allow the co-location of the virtual nodes belonging to the same VN. However, unlike the few existing works that allow virtual node co-location and consider

* Correspondence to: Software Engineering Building 306, Chongqing University, Chongqing, China.

E-mail address: wuyb@cqu.edu.cn (Y. Wu).

the host-local traffic among the virtual nodes belonging to the same VN as ‘free of charge’ [9,10], we introduce the concept of switching capacity to represent hosts’ capability of processing network packets regardless of whether the communication is with local or remote virtual nodes. Switching capacity is a limited resource, and is of practical importance especially in the virtual node co-location scenarios this paper studies.

The above new perspective brings about two main challenges. (1) The amount of consumed resources in the virtual node embedding process is no longer straightforwardly obtainable. Since pairwise bandwidth demand among virtual nodes is unknown, some substrate resources such as switching capacity and link bandwidth have to be reserved based on the ‘worse cases’ so that virtual nodes’ guaranteed bandwidth is always ensured. To this end, we provide a detailed analysis on resource reservation, which plays a critical role when performing embedding steps. (2) The resource consumption/reservation is no longer strictly additive due to virtual node co-location. For example, embedding more virtual nodes on a substrate node may cause less consumption of the substrate node’s link bandwidth. To tackle this challenge, we categorize substrate resources into additive and non-additive for the first time, and devise our algorithms accordingly.

Next, we formulate the *VPC embedding problem (VPCEP)*; the objective is to maximize time-averaged revenue. VPCEP is more complex than the multi-dimensional bin packing problem [11], thus is NP-hard. To solve VPCEP, We propose Heuristic Assisted Dueling Double Deep Q Network (HA-D3QN). HA-D3QN employs D3QN, a state-of-the-art value-based structure, for Deep Reinforcement learning (DRL). To cope with the exponentially large action space, we propose three heuristic algorithms, which deal with additive and non-additive resources differently in the embedding process, to generate a set of candidate actions. Basically, these heuristic algorithms provide additional chances if a non-additive resource becomes the bottleneck that hinders further virtual node embedding. Note that setting different parameter values can make each of these algorithms produce multiple candidate actions.

The rationales for adopting D3QN DRL are as follows. (1) RL has been proved useful when trying to optimize over a sequence of decisions and pursuit long-term benefits [4], hence is well-suited to VPCEP considering the sequence of VNs to embed and the target of long-term revenue. (2) There is no apparent rule that tells which candidate (e.g., among the totally 15 in our experiment) is superior under different circumstances, thus it becomes beneficial to exploit the significant approximation capability of deep neural networks to evaluate each candidate. (3) D3QN splits Q-values into value function and advantage function, and uses two separate neural network models, i.e., target network and evaluation network; in this way, D3QN restricts the overestimation of Q-values [12,13].

We conduct extensive simulation experiments for performance evaluation. Since there is no off-the-shelf method dedicated to VPCEP, we carefully select baselines so that the performance of HA-D3QN can be reasonably assessed. Firstly, we compare with the well-known Global Resource Capacity (GRC) method [14], which is efficient, effective, and has been adopted in some recent research for virtual network embedding [15,16]. Secondly, the individual heuristic algorithms we proposed are also used as baselines, so as to investigate the Q-value approximation capability of HA-D3QN. Thirdly, we also design a random scheme to compare with, so that the effectiveness of the action selection process of HA-D3QN can be further investigated. The experiment results show that HA-D3QN outperforms the opponents in metrics like time-averaged revenue and accept ratio, validating its effectiveness. We also examine the impact of key environment parameters, including the lifetime of virtual networks, the

amount of substrate resources, and the scale of SN. We find that the advantage of HA-D3QN largely persists under different environment settings.

Our original contributions are summarized as follows:

1. We devise a VN abstraction based on the widely used VPC model, so as to reflect the essential characteristics of tenants’ requests. Most importantly, precise inter-virtual node communication demand is assumed to be unknown. Besides, unlike the majority of existing study, we allow the co-location of virtual nodes. We propose the concept of switching capacity, and provide detailed calculation of resource consumption in the virtual node embedding process.
2. We formulate the VPCEP and propose a DRL-based framework named HA-D3QN to solve it. We analyze the additive features of different substrate resources for the first time, and propose three heuristic algorithms accordingly to generate candidate actions so as to narrow down the exponentially large action space.
3. We conduct extensive simulation experiments, where different SN sizes and resource capacities are considered. The results show that HA-D3QN outperforms the individual heuristic algorithms, a random selection algorithm, and the well-known GRC algorithm with regard to time-averaged revenue and accept ratio. Besides, the advantage of HA-D3QN persists under different environment settings.

The rest of the paper is organized as follows. Section 2 introduces the background. Section 3 describes the related work. Section 4 describes the system model and Section 5 formulates the VPCEP. Section 6 elaborates on our proposed method. Section 7 is dedicated to the simulation-based performance evaluation. Finally in Section 8, we conclude the paper.

2. Background: VPC and switching capacity

2.1. Virtual private cloud

VPC is the de facto model to build virtual networks in major cloud platforms, e.g., AWS [7], GCS [8], and Azure (although the name is Virtual Network, the functionality is similar) [17]. After creating a VPC and creating Virtual Machines (VMs) in it, layer-3 connectivity among these VMs is established. Tenants can also create subnets in a VPC to enable layer-2 connectivity among VMs. It is especially noteworthy that the bandwidth demand between any pair of VMs in a VPC are not exposed to cloud providers, who determine the embedding of VNs. Actually, cloud providers can obtain only the static information about the VMs such as CPU, memory, guaranteed bandwidth,¹ highest bandwidth, etc. From the perspective of cloud providers, guaranteed bandwidth is the key to complying with the service level agreements, thus is the focus of the VPC model.

Fig. 1 demonstrates an example VPC and the corresponding Virtual Network Request (VNR), which will be detailed in Section 4.2. In the VPC, every virtual machine is associated with computing demand (CPU cycles per second) and guaranteed bandwidth.

In recent years, Distributed Virtual Routing (DVR) emerges as a promising Infrastructure as a Service (IaaS) architecture (e.g., [18–21]), where virtual routers reside in individual servers and carry out routing functions right at the servers where the sending VMs are located. Specifically, these virtual routers receive cross-subnet packets, modify their source and destination MAC

¹ In most cases, guaranteed bandwidth is not visible to tenants.

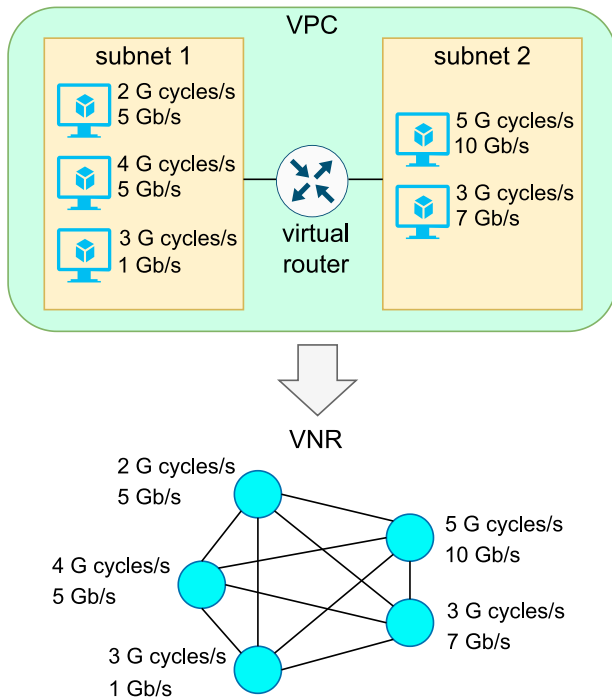


Fig. 1. A VPC with five virtual machines and the corresponding VNR.

addresses, and send them to the servers where the destination VMs are located. In this way, the performance (e.g., latency, jitter, and throughput) difference between layer-2 and layer-3 network traffic within the same VPC is largely eliminated, because the above ‘routing’ action can be conducted quickly by software such as Open vSwitch. Therefore, a VN can be depicted by a full mesh of vertices, without link weight (detailed in Section 4).

2.2. Switching capacity

In the few previous works that allow co-location of virtual nodes belonging to the same VN, host-local network traffic among those virtual nodes is considered as ‘free of charge’ [9,10]. However, this seems to be an over-simplification because in real-world cloud data centers, the capability of hosts in receiving, processing, and sending packets (measured by throughput, packet per second, etc.) is limited, regardless of whether these packets are destined to local or remote VMs.

For example, the performance of kernel based switching is mainly confined by the specific network stack implementation and the interrupt mechanism introduced by the operating system [22]. In contrast, user-space switching allows applications to directly access network interface cards, and packet processing can be accelerated on multi-core CPU. Although the need for interrupting is eliminated, the performance of user-space switching depends largely on configuration, e.g., the size of huge pages and the quantity of dedicated CPU cores [23,24]. Therefore, switching capacity is not an unlimited resource, and should be considered especially in the virtual node co-location scenarios.

3. Related work

3.1. Flat SN based work

The majority of the existing research is focused on VNEP in flat *Substrate Networks (SNs)*, hence is more suitable to applications

like peer-to-peer (P2P) and virtual private network (VPN) instead of cloud computing.

Among these works, we first introduce the non-RL approaches. Dehury and Sahoo defined the concepts of local fitness and global fitness, and proposed heuristic algorithms using these metrics to seek highest resource utilization [2]. Song et al. developed an evolutionary algorithm based on graph partitioning and integration to solve VNEP [3]. Their method outperforms other well-known metaheuristics according to the experiment results. Some works use node-ranking based heuristic algorithms, taking not only node-local but also global resources into consideration [14,25]. According to the rankings of virtual and substrate nodes, they determine the best match between the two series. Alaluna et al. considered both embedding and migration to solve VNEP [6], because a previously embedded VN may become unsuitable any more in later steps. Similarly, Cao et al. also studied the re-optimization of initial VN mappings [26]. The first embedding step fulfills only the virtual resource demand, and the second re-embedding step guarantees that the QoS demand of the VN is met. Yu et al. adopted membrane computing, which can solve NP-complete problems in polynomial time under ideal conditions [27]. They constructed a dual-parallel infrastructure over a particular membrane structure, which is able to handle a wide scope of mapping diversities. Fan et al. proposed node proximity sensing and path comprehensive evaluation for virtual network embedding [28]. Their algorithm can solve the problem of long virtual node mapping distance and unbalanced consumption of adjacent node link resources. Aitaba et al. raised a two-stage linear programming based algorithm that can significantly reduce the number of possible paths [29]. They first reduced the input data set to decrease the model complexity, then applied a combinatorial solver to determine near-optimal solutions. Elkael et al. applied Nested Rollout Policy Adaptation (NRPA), a variant of the Monte Carlo Tree Search algorithm, for virtual network embedding [30]. In their approach, a well-performing heuristic is needed for initialization of the policy. Nguyen and Huang raised a genetic algorithm based distributed parallel method [31]. They increased the effectiveness of the fitness function by introducing three novel evaluation factors. Li et al. used the metric GRC (global resource capacity) to evaluate the potential of each substrate node, and raised two functions of cost and node equilibrium to balance all the factors of VNE [16]. These works prohibit the nodes of a VN from being embedded in the same substrate node and assume that inter-virtual node communication is known.

In recent years, there emerge some RL methods. Yuan et al. engaged the famous Q-learning algorithm [4], hence is confined to only small scale problems because every state is stored in Q-table. Thakkar et al. proposed a three-stage embedding scheme [5]. The first and second stages are based on supervised learning, for determining whether a VN should be accepted, and predicting the attributes of VN. The third step directly applies the SARSA algorithm. Zhang et al. proposed combining graph convolutional neural network and RL. They raised a fitness value to deal with the changing VN topology and resource attributes [32]. Yan et al. adopted parallel policy gradient and used a graph convolutional neural network to extract feature from graph topology [33]. After training a model, the embedding decision can be made very quickly. Yao et al. assumed that an effective algorithm for historical data also works well for future request [1]. They extracted four features for each substrate node, and concatenate all the nodes’ feature vectors to build a matrix as input to a neural network, which outputs the probability for each substrate node. Then they devised a greedy algorithm based on the probabilities. Suzuki and Harada dealt with the time varying demand of VNs [34]. They introduced two types of agents, one for improving the objective function, the other for satisfying the constraints by giving large

Table 1
Literature comparison.

Reference	Algorithm	Major difference with our work
[1] Yao et al. (2018)	Node feature based greedy algorithm	VN and SN modeling, algorithm type
[2] Dehury and Sahoo (2019)	Fitness based heuristic algorithm	VN and SN modeling, algorithm type
[3] Song et al. (2020)	Evolutionary algorithm	VN and SN modeling, algorithm type
[4] Yuan et al. (2020)	Q-learning RL	VN and SN modeling, relies on (state, action) table
[5] Thakkar et al. (2020)	Supervised learning & SARSA RL	VN and SN modeling, relies on (state, action) table
[9] Duan et al. (2015)	Greedy algorithm	VN modeling, algorithm type, tenants' traffic type
[10] Dehury and Sahoo (2020)	Heuristic two-stage failure aware algorithm	VN modeling, optimization objective
[14] Gong et al. (2014)	Node ranking based heuristic algorithm	VN and SN modeling, algorithm type
[15] Wang et al. (2019)	Heuristic assisted DRL	VN and SN modeling, candidate action generation method
[16] Li et al. (2022)	GRC based heuristic algorithm	VN and SN modeling, algorithm type
[25] Cao et al. (2018)	Node ranking based heuristic algorithm	VN and SN modeling, algorithm type
[27] Yu et al. (2018)	Membrane computing algorithm	VN and SN modeling, algorithm type
[28] Fan et al. (2021)	Node proximity based heuristic algorithm	VN and SN modeling, algorithm type
[29] Ait aba et al. (2021)	Two-stage linear programming	VN and SN modeling, algorithm type
[30] Elkael et al. (2021)	Nested rollout policy adaptation	VN and SN modeling, algorithm type
[31] Nguyen and Huang (2021)	Genetic algorithm	VN and SN modeling, algorithm type
[32] Zhang et al. (2022)	Graph convolutional neural network & RL	VN and SN modeling, mainly for adapting to topology changes
[33] Yan et al. (2020)	Graph convolutional neural network & RL	VN and SN modeling, policy based RL
[34] Suzuki and Harada (2020)	Double RL agents	VN and SN modeling, mainly for dealing with varying resource demand
[35] Ballani et al. (2011)	Subtree based heuristic algorithm	VN modeling, algorithm type
[36] Duan and Yang (2016)	Greedy algorithm	VN modeling, algorithm type, tenants' traffic type

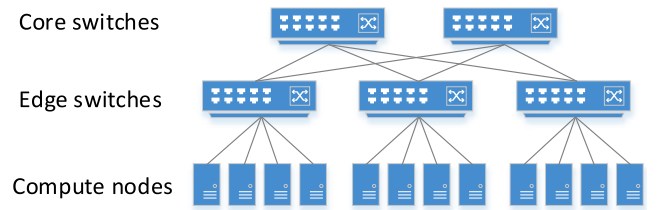
penalties if violation occurs. The work by Wang et al. also exploits temporal difference based DRL [15] to select action among a set of candidates constructed according the GRC metric. However, these approaches cannot be directly applied to our scenario because they assume flat SN structure and rely on exposed inter-virtual node bandwidth. Besides, the nodes of the same VN are not allowed to be embedded in the same substrate node.

3.2. Hierarchical SN based work

Few works concentrate on the VNEP in hierarchical SNs that represent CDC networks. The research by Ballani et al. is one of the pioneering works that study the embedding of virtual resources into hierarchical data center networks [35]. They designed two abstractions to describe VNs: one allows over-subscription, the other does not. Their proposed algorithm leverages the hierarchical structure of the SN. Specifically, it iteratively selects sub-trees in the SN to meet the demand of VN in a greedy manner. However, the method requires that cloud tenants expose part of their business pattern. Duan et al. proposed a novel virtual network abstraction, termed MVN, to represent tenants' multicast requirements, then greedily searched for the embedding method that use as few edge switches as possible [9,36]. In contrast, our paper concentrates on the east-west communication of virtual machines, instead of multicast traffic. Dehury and Sahoo studied the embedding problem in fat-tree SNs [10]. Unlike most related work, which usually takes revenue and accept ratio as objectives, their study is dedicated to minimizing the impact of potential physical resource failures. None of these works use the widely adopted VPC model for VN abstraction. Besides, although [9,10] allow virtual node co-location, they assume host-local traffic is 'free-of-charge' thus unlimited.

3.3. Summary

The related works are listed in Table 1, and we provide a summary here. (1) Existing research is mainly dedicated to scenarios where the SNs are flat, thus is hardly applicable to hierarchical SNs that represent CDC networks, mainly because while all the nodes in a flat network can host virtual nodes, only the compute nodes in a CDC network can host virtual nodes. (2) The widely adopted VN abstraction assumes the exposure of the internal communication demand among virtual nodes, which is usually not known to cloud providers. (3) The majority of the existing

**Fig. 2.** An example SN defined by (2, 3, 4).

solutions prohibit the nodes of a VN from being embedded in the same substrate node, hence not fully exploiting the virtualization capability of cloud platforms.

4. System model

4.1. Substrate network

This paper adopts the three-tier fat-tree structure [9]. Because of its good scalability, this structure can work as basic building blocks to recursively construct trees with more tiers [36]. Accordingly, an SN is determined by a tuple (m_c, m_e, m_n) , where the positive integers m_c , m_e , and m_n stand for the quantities of core switches, edge switches, and the compute nodes connected to each edge switch, respectively. A substrate link exists between any pair of edge and core switches, and no link exists between switches in the same tier. This SN structure can be viewed as a particular Clos network folded at the middle layer, and any obtained property for this structure can be recursively applied to a larger network with more tiers [36]. We show an example in Fig. 2.

We denote the sets of substrate nodes and links in SN s by N^s and L^s , respectively. Note that in this paper, the term 'substrate node' is equivalent to 'compute node' because switches are not for embedding virtual nodes thus are not regarded as 'substrate nodes'. Each compute node, $n^s \in N^s$, is associated with two attributes: (1) residual computing capacity c_{n^s} ; (2) residual switching capacity ρ_{n^s} , which determines the node's capability of processing incoming, outgoing, and internal packets, measured in Gb/s. Each substrate link $l^s \in L^s$ has a residual bandwidth b_{l^s} , also measured in Gb/s. Note that throughout this paper, we may omit the units for simpler illustration. We list the main notations in Table 2.

Table 2
Main notations.

s	An SN
N^s	The set of substrate nodes in s
n^s	A substrate node, $n^s \in N^s$
c_{n^s}	The residual computing capacity of n^s
ρ_{n^s}	The residual switching capacity of n^s
$E(n^s)$	The edge switch of n^s
$L(n^s)$	The substrate link between n^s and $E(n^s)$
L^s	The set of substrate links in s
l^s	A substrate link, $l^s \in L^s$
b_{l^s}	The residual bandwidth of l^s
E^s	The set of edge switches in s
e^s	An edge switch, $e^s \in E^s$
$U(e^s)$	The set of uplinks for e^s
k	An VNR
t^k	The arrival time of k
d^k	The lifetime of k
V^k	The set of virtual nodes in k
v^k	A virtual node, $v^k \in V^k$
μ_{v^k}	The computing demand of v^k
b_{v^k}	The guaranteed bandwidth of v^k
j	An index for the substrate nodes in a given SN
V_j^k	The set of virtual nodes that belong to k and are placed on j
Ψ_j^k	The maximum total traffic (including host-local and cross-host) bandwidth caused by V_j^k on j when the elements in V_j^k are trying their best to communicate with each other
Λ_j^k	The maximum cross-host traffic bandwidth caused by V_j^k on j when the elements in V_j^k are trying their best to communicate with the elements in $V^k - V_j^k$
Γ_j^k	The maximum host-local traffic bandwidth caused by the remaining guaranteed bandwidth in the elements of V_j^k when the elements in V_j^k are trying their best to communicate with the elements in $V^k - V_j^k$
B_j^k	The maximum summed traffic bandwidth (thus the reserved switching capacity) caused by V_j^k on j
e^s	An edge switch in SN s
N_{e^s}	The set of substrate nodes connected to e^s
$\eta(e^s)$	The maximum bilateral edge-core traffic bandwidth of e^s
$\mathbb{M}$	The mapping from virtual nodes to substrate nodes
$Rev(k)$	The revenue generated by embedding k
K_T	The set of VNRs arrived before time instance T .
$\mathbb{V}$	The value function part of the D3QN structure
$\mathbb{A}$	The advantage function part of the D3QN structure
$\mathcal{A}$	The set of candidate actions
S_t	MDP state at the beginning of time instance t
A_t	MDP action at the beginning of time instance t
R_t	MDP reward at the beginning of time instance t

The set of edge switches is denoted by E^s , and the uplink (connecting edge to core) set of edge switch $e^s \in E^s$ is denoted by $U(e^s)$. The edge switch of node n^s is represented by $E(n^s)$, and the link between n^s and $E(n^s)$ is represented by $L(n^s)$.

4.2. Virtual network request

A Virtual Network Request (VNR) represents one or more newly created VMs that belong to a newly created VPC. We model VNR k as the full mesh of a set of virtual nodes V^k . The full mesh indicates connectivity between any pair of virtual nodes. The relative arrival time and lifetime of k are denoted by non-negative integer t^k and positive integer d^k , respectively. Each virtual node, $v^k \in V^k$, is associated with two attributes: computing demand μ_{v^k} , and guaranteed bandwidth b_{v^k} , which is the minimum allowed bandwidth for incoming and outgoing traffic combined. Maintaining and ensuring guaranteed bandwidth for virtual machines is important for cloud providers because they effectively avoid bandwidth starvation even under fierce resource competition. An example VNR is displayed in Fig. 1.

Implications: Because of the full mesh structure of VNR and the fact that VM-to-VM communication demand is not known in advance in VNR, the switching capacity and bandwidth resources in substrate nodes and links cannot be allocated based on precise demand. Instead, we need to reserve these resources considering the worst cases. The reservation details will be elaborated on in the following two subsections.

4.3. The reservation of switching capacity and link bandwidth

On a given substrate node, the occupied/reserved switching capacity is the summed bandwidth of the incoming, outgoing, and internal network traffic on this node. Given request k and substrate node n_j^s (in the following, we may directly use the variable j to denote this substrate node for clearer illustration), V_j^k denotes the set of virtual nodes that belong to k and are placed on n_j^s . We use B_j^k to denote the reserved switching capacity caused by V_j^k on n_j^s .

Suppose the virtual nodes in VNR k are divided into two groups, i.e., V_j^k and $V^k - V_j^k$. When calculating B_j^k , we assume the nodes in $V^k - V_j^k$ are already embedded in some substrate nodes other than n_j^s . We consider two extreme cases that determine the bounds on the value of B_j^k : (1) The virtual nodes in V_j^k are trying their best to communicate with each other, and they communicate with the nodes in $V^k - V_j^k$ only if they have extra guaranteed bandwidth that cannot be exploited for host-local communication. Let Ψ_j^k denote the maximum total traffic (including host-local and cross-host) bandwidth in n_j^s caused by V_j^k in this case. (2) The virtual nodes in V_j^k are trying their best to communicate with the nodes in $V^k - V_j^k$. Only when any element in V_j^k has extra guaranteed bandwidth that cannot be exploited for cross-host communication will it communicate with the elements in V_j^k . We use Λ_j^k to represent the maximum bilateral cross-host (i.e., between n_j^s and its edge $E(n_j^s)$) traffic bandwidth caused by V_j^k on n_j^s , and Γ_j^k to denote the maximum host-local traffic bandwidth caused by the remaining guaranteed bandwidth of the elements in V_j^k .

Using the above notations, we calculate B_j^k by:

$$B_j^k = \max\{\Psi_j^k, \Lambda_j^k + \Gamma_j^k\}. \quad (1)$$

Note that we separate cross-host consumption (Λ_j^k) and host-local consumption (Γ_j^k) for the second extreme case because Λ_j^k is the occupied link bandwidth and will be used in the constraint definitions in Section 5

Next, we provide the detailed calculation of Ψ_j^k , Λ_j^k , and Γ_j^k .

Proposition 1. The maximum traffic bandwidth in n_j^s caused by best-effort host-local communication among the virtual nodes in V_j^k is:

$$\Psi_j^k = \begin{cases} \sum_{v \in V_j^k} b_v / 2 & \text{if } \max_{v \in V_j^k} b_v \leq \sum_{v \in V_j^k} b_v / 2, \\ \sum_{v \in V_j^k} b_v - \max_{v \in V_j^k} b_v + \min\left\{ \sum_{v \in (V^k - V_j^k)} b_v, \right. & \\ \left. 2 \max_{v \in V_j^k} b_v - \sum_{v \in V_j^k} b_v \right\} & \text{else.} \end{cases} \quad (2)$$

Proof (In Appendix A.). $\square$

Apparently, the maximum cross-host bandwidth caused by V_j^k , denoted by Λ_j^k , in this extreme case is as follows.

$$\Lambda_j^k = \min\left\{ \sum_{v \in V_j^k} b_v, \sum_{v \in (V^k - V_j^k)} b_v \right\}. \quad (3)$$

If $\sum_{v \in V_j^k} b_v \leq \sum_{v \in (V^k - V_j^k)} b_v$, then no extra bandwidth remains, we get $\Gamma_j^k = 0$. Otherwise, in order to obtain Γ_j^k , we first define a particular subset of V_j^k :

$$V_j'^k = \{v | v \in V_j^k, b_v > (\sum_{v \in V_j^k} b_v - \sum_{v' \in (V^k - V_j^k)} b_{v'})/2\}, \quad (4)$$

where $\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v$ is the remaining bandwidth for host-local communication. The intuition is that the guaranteed bandwidths of the elements in $V_j'^k$ are so large that they may not be used up for cross-host communication. To further investigate, we define an intermediate variable b'_v for every element in $V_j'^k$:

$$b'_v = b_v - (\sum_{v \in V_j^k} b_v - \sum_{v' \in (V^k - V_j^k)} b_{v'})/2, \quad (5)$$

then obtain Γ_j^k as follows.

Proposition 2. Given

$$\sum_{v \in V_j^k} b_v > \sum_{v \in (V^k - V_j^k)} b_v, \quad (6)$$

Γ_j^k is determined by:

$$\Gamma_j^k = \begin{cases} (\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v)/2 & \text{if } \sum_{v \in V_j'^k} b'_v \leq \sum_{v \in (V^k - V_j^k)} b_v, \\ \sum_{v \in V_j^k} b_v - \max_{v \in V_j^k} b_v & \text{else.} \end{cases} \quad (7)$$

Proof (In Appendix B.). $\square$

4.4. Edge-core traffic

In order to define the problem constraints, we calculate the maximum traffic bandwidth from edge switches to core switches. For an edge switch e^s in SN s , we denote the set of substrate nodes connected to e^s by:

$$N_{e^s} \triangleq \{n^s | n^s \in N^s, E(n^s) = e^s\}. \quad (8)$$

Given VNR k , the maximum bilateral edge-core traffic bandwidth of e^s is calculated by:

$$\eta(e^s) = \min\{\sum_{n^s \in N_{e^s}} \sum_{v \in V_{n^s}^k} b_v, \sum_{n^s \in (N^s - N_{e^s})} \sum_{v \in V_{n^s}^k} b_v\}. \quad (9)$$

Intuitively, $\eta(e^s)$ is the smaller one between the summed guaranteed bandwidths of the substrate nodes that belong to e^s and that of the other substrate nodes.

4.5. Categorization of substrate resources

Suppose a virtual node in a given VNR occupies a certain resource, and embedding this virtual node in a substrate node causes more consumption of this resource on the substrate node, this resource is strict-additive. If embedding the virtual node in the substrate node causes no less consumption of the resource on this substrate node, the resource is additive. Note that strict-additive is a special case of additive. Finally, if a resource is not additive, we categorize it as non-additive.

Obviously, computing capacity is strict-additive, because the additional consumption by a newly embedded virtual node is the

exact amount demanded by the node. In contrast, link bandwidth is apparently non-additive, which can be proved by a simple example: while embedding the virtual nodes of a VPC in different substrate nodes causes cross-host communication that consumes link bandwidth, embedding these virtual nodes together in the same substrate node does not occupy any link bandwidth.

Unlike the above two resources, the categorization of switching capacity is not straightforwardly obtainable. Next, we prove switching capacity is additive.

Theorem 1. Switching capacity is an additive resource.

Proof (In Appendix C.). $\square$

Implications: The problem of embedding virtual nodes with different demand in substrate nodes with different capacities is essentially a vector bin packing problem, which has been extensively researched and promising approaches have been raised. However, traditional solutions for vector bin packing implicitly assume additive resources [11,37], hence are hardly applicable to the scenario this paper is concentrated on. To address the issue and solve VPCEP, we propose three candidate action generation algorithms (in Section 6.2) that deal with non-additive resources differently than additive resources. Specifically, we allow additional chances of embedding if a non-additive resource is about to be ‘over-occupied’, considering that embedding more virtual nodes may lead to less resource occupation. We put the study of dealing with strict-additive and other additive resources differently as future work.

5. The VPC embedding problem

For SN s and VNR k , virtual network embedding is defined by a mapping: $\mathbb{M} : V^k \rightarrow N^s$. The virtual nodes in k can be mapped to the same substrate node. The target of deciding on a mapping is to efficiently utilize the substrate resources so as to generate as much revenue as possible. Inspired by the model adopted by Wang et al. [15], we define revenue as follows:

$$Rev(k) \triangleq \begin{cases} w_1 \sum_{v \in V^k} \mu_v + w_2 \sum_{v \in V^k} b_v & \text{if } k \text{ is accepted,} \\ 0 & \text{if } k \text{ is blocked,} \end{cases} \quad (10)$$

where w_1 and w_2 are the unit prices charged for computing demand (μ_v) and bandwidth demand (b_v), respectively.

We define the objective as the maximized long term time-averaged revenue:

$$\lim_{T \rightarrow \infty} \frac{\sum_{k \in K_T} Rev(k)}{T}, \quad (11)$$

where K_T represents the set of VNRs arrived before time instance T .

The constraints on computing capacity and switching capacity are:

$$\sum_{v \in V_j^k} \mu_v \leq c_{n_j^s} \quad (\forall n_j^s \in N^s), \quad (12)$$

$$B_j^k \leq \rho_{n_j^s} \quad (\forall n_j^s \in N^s). \quad (13)$$

The constraints on server-edge and edge-core bandwidth are:

$$\Lambda_j^k \leq b_{L(n_j^s)} \quad (\forall n_j^s \in N^s), \quad (14)$$

$$\eta(e^s) \leq \sum_{l \in U(e^s)} b_l \quad (\forall e^s \in E^s). \quad (15)$$

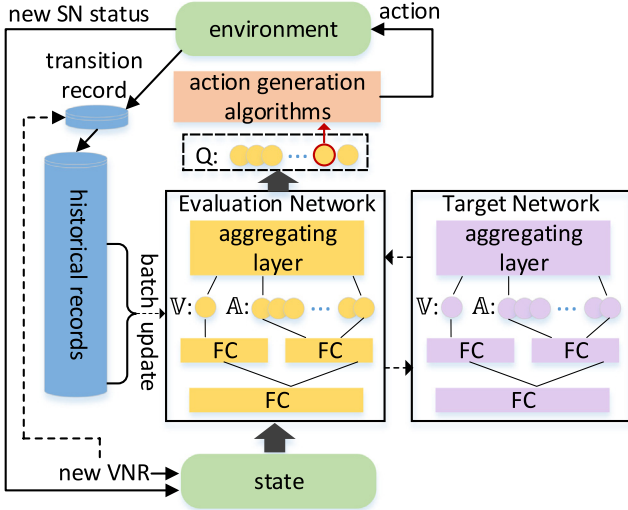


Fig. 3. Overview of our framework.

This paper does not consider network latency as a constraint because in real-world CDCs, the intra-DC latency is largely confined by the limited hops between any pair of compute nodes. According to the measurement study by Popescu and Moore on four public cloud data centers, intra-DC latency almost always stays within one millisecond [38].

VPCEP is formulated as:

$$\max \quad (11)$$

$$\text{s.t.} \quad (12), (13), (14), (15)$$

6. HA-D3QN

Fig. 3 presents an overview of our method HA-D3QN. The core is the DRL component, which adopts Dueling Double Deep Q Network (D3QN). D3QN combines Double DQN [12] and Dueling DQN [13]. To eliminate the need for storing the Q-values of all state-action pairs, deep neural networks are used for approximating the Q-values of state-action pairs. D3QN splits the Q-values into two different parts, i.e., the value function (denoted by $\mathbb{V}$) and the advantage function (denoted by $\mathbb{A}$). By using two separate neural network models, i.e., target network and evaluation network, double DQN reduces the overestimation of Q-values. The evaluation network is for training the parameters, and the target network is for calculating the Q value of the next state. The parameters of the evaluation network are periodically copied to the target network.

To address the issue of exponentially large action space, the DRL agent relies on dedicated heuristic algorithms to generate candidate actions, the number of which determines the number of Q-values outputted by the D3QN architecture. After conducting the action with the highest Q-value, the SN changes to a new status, and by combining this status with a newly arrived VNR, we get a new state for the next iteration. Note that in each iteration, only the selected candidate (algorithm-parameter pair) will be conducted. The agent uses epsilon-greedy method at the beginning and gradually decreases the probability of exploration to zero. At the end of each iteration, we randomly select a batch of historical state transition records to train the neural network based on temporal difference (TD) learning [39].

This framework is extensible because new action generation algorithms and modifications to existing algorithms are allowed.

6.1. D3QN based reinforcement learning

To fit our solution to the RL model, we define state, action, and reward as follows. Given an SN s , we define the state at time instance t , S_t , as the combination of the residual substrate resource of each type and the demand of the VNR (denoted by k) in t . Formally, $S_t = \{(c_n, \rho_n) | n \in N^s\} \cup \{b_l | l \in L^s\} \cup \{(\mu_v, b_v) | v \in V^k\}$. Note that we assume a one-to-one correspondence between time instance t and VNR k , which means a separate iteration for each newly arrived VNR. We define action A_t as a mapping $\mathbb{M} : V^k \rightarrow N^s$, such that the resource constraints are satisfied. After taking an action, the system receives a reward R_{t+1} , and changes its state to S_{t+1} . By taking into account our optimization objective, we directly set the reward to the revenue for embedding k , i.e., $R_{t+1} = Rev(k)$.

We define the Q-value function of a state-action pair as the expected discounted cumulative reward:

$$Q^\pi(S_t, A_t) \triangleq \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i R_{t+i+1} | S_t, A_t \right], \quad (16)$$

where π is the policy followed by the agent, $\gamma \in [0, 1]$ is a discount factor. Then the optimal Q-value function is defined as:

$$Q^*(S_t, A_t) \triangleq \max_\pi Q^\pi(S_t, A_t), \quad (17)$$

which satisfies the Bellman equation:

$$Q^*(S_t, A_t) = \mathbb{E}_{S'_{t+1}} [R_{t+1} + \gamma \max_{A_{t+1}} Q^*(S'_{t+1}, A_{t+1})]. \quad (18)$$

As stated previously, we adopt Dueling Double Deep Q Network (D3QN), which is the combination of Double DQN and Dueling DQN. The D3QN network provides separate estimates of the value ($\mathbb{V}$) and advantage ($\mathbb{A}$) functions, and aggregates the two streams to produce a Q value for each candidate action according to:

$$Q(S, A, \theta, \varphi, \phi) = \mathbb{V}(S, \theta, \varphi) + \mathbb{A}(S, A, \theta, \phi) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} \mathbb{A}(S, a', \theta, \phi), \quad (19)$$

where θ denotes the network parameters of the public part, φ denotes the parameters of $\mathbb{V}$, and ϕ denotes the parameters of $\mathbb{A}$. $\mathcal{A}$ represents the set of candidate actions.

To update the parameters for D3QN, we store the state transition records in a first-in-first-out (FIFO) in-memory storage with fixed size. After each iteration, we randomly select a batch of historical records, and calculate the target for each one of them. For record $(S_t, A_t, R_{t+1}, S_{t+1})$, target y_t is obtained by:

$$y_t = R_{t+1} + \gamma Q^{\text{target}}(S_{t+1}, \arg \max_{a' \in \mathcal{A}} Q^{\text{eval}}(S_{t+1}, a', \omega), \omega'), \quad (20)$$

where Q^{target} and Q^{eval} denote the Q-value functions of the target and evaluation networks, respectively (D3QN uses a double-network architecture). ω' and ω denote the parameters of the target and evaluation networks, respectively, and ω' is set to ω periodically [12]. The loss function is defined as follows:

$$L_t(\omega) = \frac{1}{|T_{\text{batch}}|} \sum_{t \in T_{\text{batch}}} (y_t - Q^{\text{eval}}(S_t, A_t, \omega))^2, \quad (21)$$

where T_{batch} is the set of time instances in the batch. Then we update ω by gradient descent [39].

6.2. Algorithms for candidate action generation

We propose three built-in action generation algorithms. By using different parameters, each algorithm is able to generate multiple actions.

Algorithm 1: VNR Closeness First (VCF)

Input : $vl_c, vl_b, elist, nlist, \mathbb{M}$
Output: $\mathbb{M}$

```

1 partial mapping  $m = \{ \}$ 
2 for  $e$  in  $elist$  do
3   for  $n$  in  $nlist(e)$  do
4     for  $v$  in  $vl\_c$  do
5       if  $v$  can be embedded in  $n$  then
6          $m.add(< v, n >)$ ,  $vl\_c.del(v)$ ,  $vl\_b.del(v)$ 
7       else if failed due to  $c_n$  or  $\rho_n$  then
8         break
9       else
10         $m = \text{handleBW}(vl\_b, n, m)$ 
11        update  $vl\_c$  and  $vl\_b$ 
12      endfor
13    endfor
14     $\mathbb{M}.extend(m)$ ,  $m = \{ \}$ 
15  endfor
16 return  $\mathbb{M}$ 

```

6.2.1. VNR closeness first (VCF)

Firstly, we introduce the VNR Closeness First (VCF) algorithm, which aims to embed the virtual nodes that belong to the same VNR as close as possible, so as to consume less overall resource and leave more resource for future embedding. We sort and store virtual nodes in two lists: vl_c is in descending order of computing demand, and vl_b is in ascending order of guaranteed bandwidth. Inspired by Gong et al. [14], we weight each compute node as follows:

$$W(n) = h_1 c'_n + h_2 \rho'_n + (1 - h_1 - h_2) b'_{l(n)} \quad (n \in N^s), \quad (22)$$

where $h_1 \in [0, 1]$, $h_2 \in [0, 1]$ are weight parameters, and c'_n , ρ'_n , and $b'_{l(n)}$ are normalized residual computing capacity, switching capacity, and link bandwidth, respectively. Here 'normalized' means that $\sum_{n \in N^s} c'_n = 1$, $\sum_{n \in N^s} \rho'_n = 1$, and $\sum_{l \in L^s} b'_l = 1$.

We sort the edge switches in substrate network s by the summed weight of their connected compute nodes in descending order, and get a switch list $elist(s)$. For each edge switch e , we also sort its connected compute nodes by their weight in descending order, and get a node list $nlist(e)$. These lists along with $\mathbb{M}$, a global variable recording the mapping/embedding, are the input to the algorithm. When the algorithm terminates, if all the elements in V^k exist in $\mathbb{M}$, we get a feasible mapping. The pseudo code is presented in Algorithm 1, where m stores current partial mapping.

If insufficiency occurs for computing capacity and switching capacity, which are additive resources, we simply turn to the next substrate node (line 7–8). However for insufficient link bandwidth, which is non-additive, we define a function $\text{handleBW}()$ (Algorithm 2), where we iteratively split vl_b into two parts so that the size (denoted by i) of the left part ($vl_b[i-1]$) gradually increases. According to Eq. (3), as more and more virtual nodes in V^k are embedded in n , the demand on link bandwidth first increases, then decreases, finally to zero. Therefore, the iteration continues if only the bandwidth constraint is violated, because embedding more virtual nodes may result in less bandwidth consumption. The complexity of VCF algorithm is $O(|N^s| |V^k|^2)$, where $|N^s| |V^k|$ is due to the traversal of the virtual and substrate nodes, and the additional $|V^k|$ is due to the processing in $\text{handleBW}()$.

There are two potential issues associated with VCF. First, while computing capacity and switching capacity can be efficiently consumed, some bandwidth resources may become largely empty and cannot be used for other VPCs (large island problem). Second, VCF may cause serious fragmentation, thus is less capable of

Algorithm 2: Function $\text{handleBW}()$

Input : vl_b, n, m
Output: m

```

1  $tmp\_vl = [ ]$ 
2 for  $i = 0, i \leq \text{len}(vl\_b), i++$  do
3   if cannot embed  $vl\_b[i-1]$  in  $n$  then
4     if failed due to  $c_n$  or  $\rho_n$  then
5       break
6     else if failed due to bandwidth then
7       continue
8     else if  $\text{len}(vl\_b[i-1]) > 0$  then
9        $tmp\_vl = vl\_b[i-1]$ 
10  endfor
11 if  $tmp\_vnl \neq [ ]$  then
12   embed  $tmp\_vl$  in  $n$ , update  $m$ 
13 return  $m$ 

```

accepting future large requests. To cope with the two issues, we also propose OCF, described below.

6.2.2. Overall closeness first (OCF)

In contrast to VCF, which tries large residual substrate resource and large demand first to increase the possibility of successfully embedding large virtual nodes, the Overall Closeness First (OCF) algorithm targets filling in the small fragments first so that the overall residual resource is less fragmented thus more capable of accepting future large unsplitable requests. OCF differs with VCF in two ways: (i) vl_c is in ascending order of computing demand; and (ii) $elist$ and $nlist$ are in ascending order of weight. Other than those, the logic of the two algorithms are the same, and the complexity of OCF algorithm is also $O(|N^s| |V^k|^2)$.

6.2.3. Probabilistic round robin (PRR)

The Probabilistic Round Robin (PRR) algorithm works as follows. While vl_b is still in ascending order of bandwidth, vl_c , $elist$, and $nlist$ are in random order. The compute nodes form a 'ring', so that each node can be traversed multiple times. The algorithm iterates over the compute nodes one by one, and embed virtual nodes in it with probability $p(z) = v^z$ (if the substrate resources are adequate). $v \in [0, 1]$ is a discount factor, and z is the quantity of virtual nodes that have been continuously embedded in this substrate node. If link bandwidth violation occurs, we also call $\text{handleBW}()$ to find a set of feasible virtual nodes, and embed all of them with probability $p(z)$. The algorithm terminates and reports a failure if every compute node is tried once but failed continuously. The complexity of PRR algorithm is also $O(|N^s| |V^k|^2)$.

7. Evaluation

7.1. Experiment setup

Like many previous works in this field [5,14–16,25,28–30,33], we conduct simulation experiments to validate the performance of our approach. We create an SN with size (3, 6, 12) (the parameters are introduced in Section 4.1), so there are 72 substrate nodes in total. We set the computing capacity, the switching capacity of the substrate nodes, and the bandwidth of server-edge links, to random values within [20, 50]. The bandwidth of edge-core link is set to a fixed value 300. For computing capacity, the unit is G cycles/s; for bandwidth and switching capacity, the unit is Gb/s. Note that we omit these units in the following contents for simpler illustration.

VNRs arrive one by one, and for each one of them, the quantity of virtual nodes follows a uniform distribution between 2 and

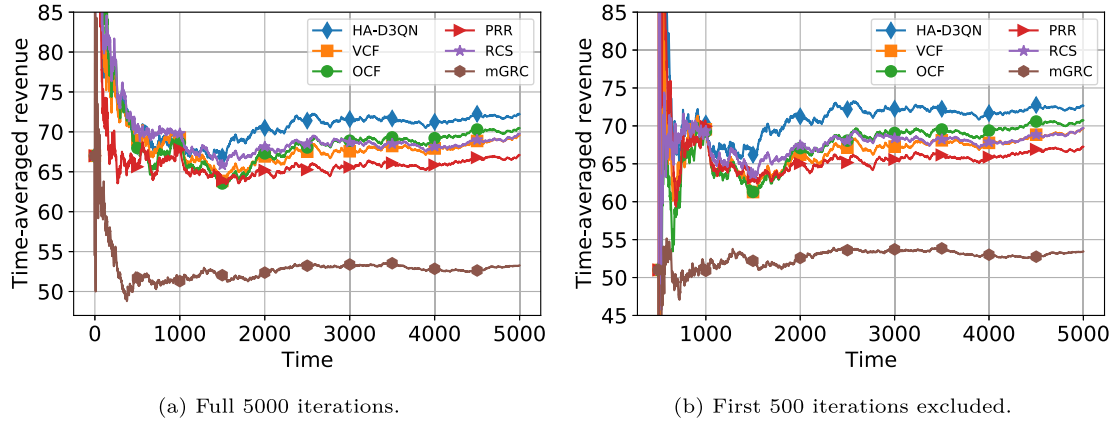


Fig. 4. Time-averaged revenue.

20. The computing demand and guaranteed bandwidth of each virtual node follow a uniform distribution with range [5, 10]. The lifetime of each VNR, the impact of which will be investigated later, conforms to an exponential distribution with average 50. We set both of the unit prices in Eq. (10) to 1, and the discount factor in Eq. (16) to 0.99.

For the D3QN structure, we set the quantities of nodes in the public layer and the function layers (i.e., ∇ and Δ) to 128 and 64, respectively. The activation function is ReLU. The size of the historical record repository is set to 1500, and the batch size for training is set to 128.

We use 5 pairs of parameters: (0.4, 0.4), (0.3, 0.3), (0.5, 0.3), (0.3, 0.5), and (0.4, 0.2) for the VCF and OCF algorithms (formula (22)). For the PRR algorithm, we set the discount factor ν to 0.7, and by randomly ordering νl_c , $elist$, and $nlist$, we get 5 more candidates. To sum up, these algorithms and parameters form at most 15 candidate actions for each VNR.

The experiment is conducted on a server with Intel Xeon 2687 W v4 12core CPU and Nvidia Titan Xp 12gb GPU. The simulation environment and the heuristic algorithms are implemented by Java programming language, and the deep learning part is implemented with PyTorch. The connection between Java and PyTorch is accomplished with the DeepJavaLibrary.

7.2. Compared methods

As far as we know, there is no off-the-shelf method dedicated to VPC embedding in hierarchical CDCs where virtual node co-location is allowed. The comparison is devised with two main goals in mind. (1) We compare with a well-known and widely used virtual network embedding algorithm, so that the key innovation of our method, i.e., the differentiated dealing with additive and non-additive resources, can be assessed. (2) We compare with the individual heuristic algorithms, which are proposed for candidate action generation, so that the deep reinforcement learning component of our method can be assessed. In this way, the performance of HA-D3QN can be reasonably evaluated.

We select Global Resource Capacity (GRC) algorithm [14], a well-known solution for VNEP and has been adopted in some recent works [15,16], as the baseline. Specifically, the GRC of a virtual/substrate node is a weighted summation of the resources in the entire virtual/substrate network. Intuitively, GRC quantifies the demand of each virtual node and the embedding potential of each substrate node. After obtaining the GRC vector for the SN and VNR, the algorithm sorts the nodes of the two networks in descending order according to their GRC values, and maps the nodes one by one. In this way, virtual nodes with large GRC are usually mapped to substrate nodes with large GRC. Note

that we modify GRC algorithm to suit VPCEP. We extend the computing resource in the capacity definition of GRC by including the switching capacity (first introduced in this paper), and assign 50% weight to each one of them. We redefine the neighbor of substrate node as the nodes under the same edge switch, considering that GRC is for flat SNs. The bandwidth between a pair of neighboring substrate nodes is redefined as the smaller one of these nodes' server-edge bandwidths. Finally, we set the damping factor d of GRC to 0.85, the same as [14]. We call the modified version mGRC.

In addition, we also conduct comparison with the three individual action generation algorithms proposed in this paper, i.e., VCF, OCF, and PRR. The purpose is to verify the effectiveness of the Q-value evaluation process of HA-D3QN. In other words, we investigate whether selecting the best action among a set of candidates performs consistently better than individual heuristic algorithms.

To further validate the effectiveness of the adopted D3QN structure in evaluating actions, we also introduce the Random Candidate Selection (RCS) scheme for comparison. Unlike D3QN, which selects the most promising action, RCS randomly selects an action from the same set of candidates.

7.3. Evaluation results

7.3.1. Time-averaged revenue

Fig. 4(a) demonstrates the time-averaged revenue as a function of time. The parameters for the compared VCF and OCF algorithms are (0.4, 0.4), considering that (0.4, 0.4) – VCF and (0.4, 0.4) – OCF are selected the most times by HA-D3QN in the corresponding category. As stated previously, we assume a one-to-one correspondence between time instance t and VNR k , so the x -axis also represents the quantity of iterations.

Compared with Fig. 4(a), which includes the complete 5000 iterations in our experiment, Fig. 4(b) excluded the first 500 iterations when calculating the time-averaged revenue. Since the RL agent learns the best policy by interacting with the environment, its initial performance can be rather poor. Therefore, including the initial results will inevitably lead to under-estimating the performance of HA-D3QN.

Actually, it is notable that the advantage of HA-D3QN in Fig. 4(b) is generally larger than that in Fig. 4(a). The results indicate that pre-training the model is beneficial in real deployment. Considering this, in the remaining experiments, we treat the first 500 iterations as the pre-training process and exclude them when calculating and showing the results.

As is evident in these figures, HA-D3QN outperforms the other methods, verifying its effectiveness. In particular, HA-D3QN outperforming RCS and the individual heuristic algorithms indicates

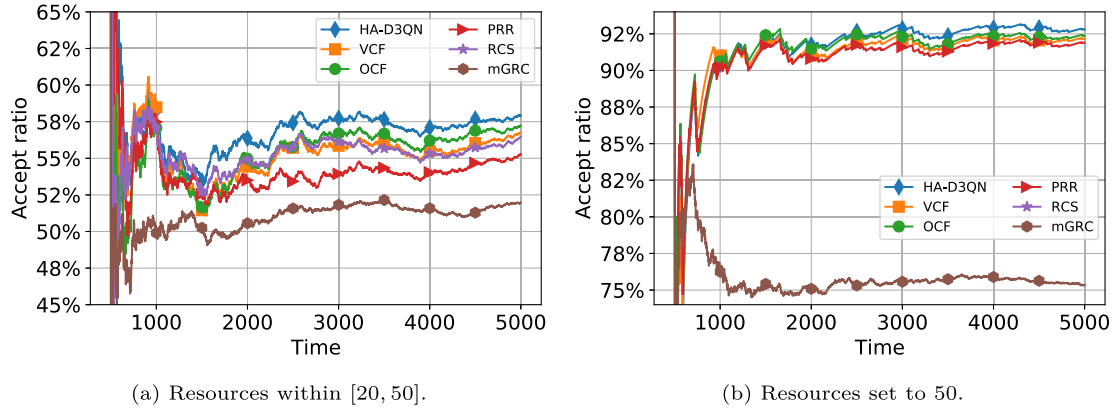


Fig. 5. Accept ratio.

that the D3QN structure is effective in evaluating each state-action' potential of meeting future demand and bringing about larger revenue.

The figures also show that mGRC performs the worst, likely because mGRC is only good at dealing with additive substrate resources, like most traditional methods do, while our scenario treats link bandwidth a non-additive resource. Among the heuristic algorithms, VCF performs the best in general, likely because it saves more substrate resources for future embedding. However, note that VCF is not always the best choice in every round, and we will elaborate on this later.

7.3.2. Accept ratio

In addition to time-averaged revenue, we also analyze *accept ratio*, i.e., the ratio of VNRs that are successfully embedded. Fig. 5(a) demonstrates accept ratio as a function of time. Generally speaking, while the relative performance of different methods persists, the difference shrinks. By comparing Figs. 4(b) and 5(a), we observe that HA-D3QN is more capable of accepting large VNRs, likely because our objective (and the reward function) is for larger revenue instead of higher accept ratio. For example, at the 2500th and 3500th iterations, while the accept ratios of HA-D3QN are 11.34% and 10.54% higher than those of mGRC, respectively, the advantages rise to 34.67% and 34.19% when comparing time-averaged revenue.

Next, we analyze whether different substrate resource densities affect the result. Fig. 5(b) shows the accept ratio of these methods when using more relaxed substrate resource settings, that is, instead of conforming to the [20, 50] uniform distribution as specified in Section 7.1, the capacities of the three types of substrate resources are directly set to 50. Apparently, while the advantage of HA-D3QN over mGRC is still significant, the difference between HA-D3QN and the individual heuristic algorithms becomes smaller, and the advantage of HA-D3QN becomes notable after around 2500 iterations. This observation indicates that the richer the underlying resources, the less beneficial to deploy the RL framework. In other words, directly running one of the heuristic algorithms yields good enough result.

7.3.3. Selected actions

We plot the selected actions from the 3501th iteration to the 4500th iteration in Fig. 6. Within this period, (0.4, 0.4)-VCF is selected 39.5% of the time, which is the highest among all the candidates. The percentages for the VCF, OCF, and PRR algorithms are 65.3%, 19.6%, and 15.1%, respectively.

In terms of the complete 5000 iterations, the VCF, OCF, and PRR algorithms account for 62%, 13.9%, and 24.1%, respectively, and (0.4, 0.4)-VCF is also selected the most times, accounting for 42%. The results comply with the trend shown in Fig. 4(b),

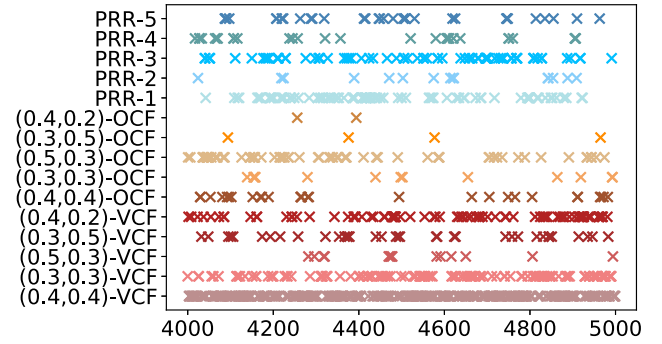


Fig. 6. Selected Actions.

that is, as time goes by, the performance of the OCF algorithm increases, along with the tendency that HA-D3QN selects OCF more often. The results also validate the fundamental idea behind our framework, that is, by selecting the best action, HA-D3QN out-performs individual heuristic algorithms.

7.3.4. Impact of parameters

In Fig. 7, we illustrate how the means and standard deviations of time-averaged revenue change with different parameter settings, including mean VNR lifetime, SN resource supply (i.e., computing capacity, switching capacity, and link bandwidth; compared with the baseline introduced in Section 7.1), and SN size. The results shown in these figures are similar. That is, generally speaking, as substrate resources become more scarce, the time-averaged revenue decreases, because it becomes harder for the SN to host virtual nodes. As is evident from these figures, the advantage of HA-D3QN, compared with RCS and mGRC, persists under different parameter settings.

7.3.5. Execution time

We plot in Fig. 8(a) the cumulative distributions of the execution time in each round of iteration for HA-D3QN and mGRC. As is evident in the figure, HA-D3QN is slower than mGRC in the majority of the rounds. Considering the start-up of a new VM usually takes tens of seconds [40], the decision time for placing the VM seems to be acceptable for both HA-D3QN and mGRC. To further investigate, we plot in Fig. 8(b) the cumulative distributions of the time needed to generate the action when different types of heuristic algorithms are selected (note that in each iteration, only the selected heuristic algorithm will be conducted). By comparing the two figures, we find that the time consumption of HA-D3QN is mainly caused by the action generation step. Actually, the deep neural network in HA-D3QN takes only 1.1 ms on average to select an action.

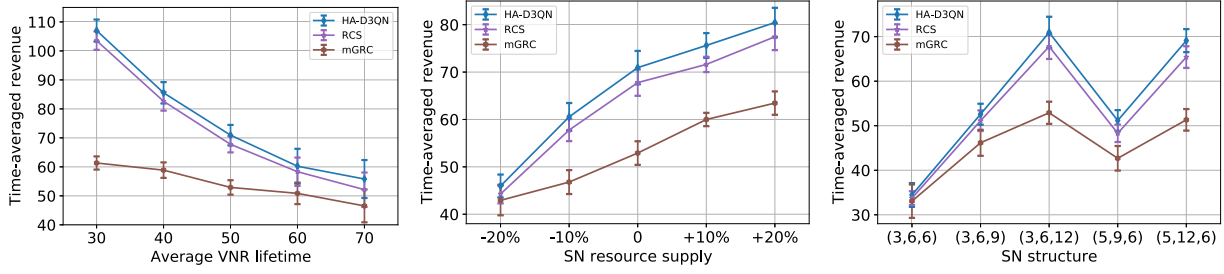


Fig. 7. Impact of parameters.

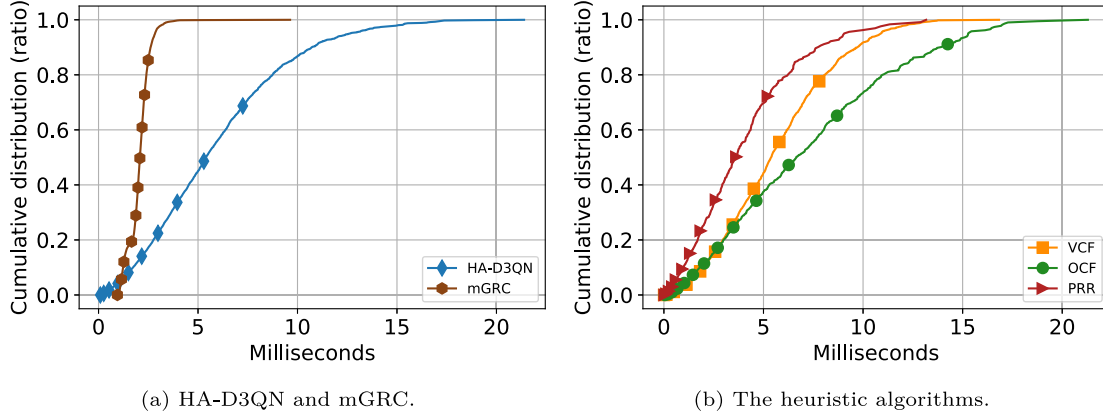


Fig. 8. The cumulative distributions of execution time.

8. Conclusion

This paper solves a virtual network embedding problem in CDC considering real-world cloud computing scenarios. We propose a VN abstraction based on VPC, categorize substrate resources according to their additive features, investigate the necessary resource reservation, and devise a D3QN based solution framework, named HA-D3QN, to solve the problem. We propose three action generation algorithms to narrow down the action space, where the additive features of different substrate resources are taken into consideration. The results of the simulation experiments verify the effectiveness of our method. Our work can be helpful in CDC operation and management.

CRedit authorship contribution statement

Meng Chen: Methodology, Validation, Investigation, Writing – original draft, Formal analysis. **Jiaxin Hou:** Methodology, Software. **Yongpan Sheng:** Methodology, Conceptualization. **Yingbo Wu:** Resources, Supervision. **Sen Wang:** Investigation, Software. **Jianyuan Lu:** Resources, Validation. **Qilin Fan:** Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work was supported by the Ministry of Education (CN) Industry-University-Research Innovation Fund (2021ALA01005), National Natural Science Foundation of China (62202075), and General Program of Chongqing Science & Technology Commission (CSTB2022TIAD-GPX0017).

Appendix A. Proof of Proposition 1

The first case in formula (2) means that if the maximum guaranteed bandwidth of the elements in V_j^k , i.e., $\max_{v \in V_j^k} b_v$, is no larger than the combined guaranteed bandwidth of the other virtual nodes in V_j^k , then all the virtual nodes in V_j^k can communicate with each other in full bandwidth, resulting in the traffic bandwidth being half of the total combined guaranteed bandwidth of these nodes.

For the second case in formula (2), we need to consider the cross-host traffic caused by the remaining guaranteed bandwidth in $\max_{v \in V_j^k} b_v$, which cannot be fully used for host-local communication. If the remaining bandwidth, which is calculated by

$$\max_{v \in V_j^k} b_v - \left(\sum_{v \in V_j^k} b_v - \max_{v \in V_j^k} b_v \right)$$

(we use X_2 to represent it) is larger than the outside communication demand

$$\sum_{v \in (V^k - V_j^k)} b_v$$

(we use X_1 to represent it). Then the maximum cross-host traffic is equal to X_1 , otherwise it is equal to X_2 .

So we simply add the smaller one of X_1 and X_2 to the host-local traffic, which is calculated by

$$\sum_{v \in V_j^k} b_v - \max_{\forall v \in V_j^k} b_v,$$

resulting in the second case of formula (2).

Appendix B. Proof of Proposition 2

The condition for the first case in formula (7) means that in V_j^k , the virtual nodes with large guaranteed bandwidth can conduct cross-host communication so that the remaining bandwidth, which is calculated by

$$\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v,$$

can be fully used for host-local communication.

For the second case in formula (7), we first prove set V_j^k contains at most one element. Let $X = |V_j^k|$, then the summation of the nodes' bandwidth in V_j^k is

$$\sum_{v \in V_j^k} b_v = \sum_{v \in V_j^k} b'_v + X \left(\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v \right) / 2.$$

According to the condition of the second case, i.e.,

$$\sum_{v \in V_j^k} b'_v > \sum_{v \in (V^k - V_j^k)} b_v,$$

we get

$$\sum_{v \in V_j^k} b_v >$$

$$\sum_{v \in (V^k - V_j^k)} b_v + X \left(\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v \right) / 2,$$

and the right part of it equals to

$$X \sum_{v \in V_j^k} b_v / 2 + (1 - X/2) \sum_{v \in (V^k - V_j^k)} b_v.$$

Since apparently $\sum_{v \in V_j^k} b_v \geq \sum_{v \in V_j^k} b_v$, we get

$$\sum_{v \in V_j^k} b_v > X \sum_{v \in V_j^k} b_v / 2 + (1 - X/2) \sum_{v \in (V^k - V_j^k)} b_v,$$

which is equivalent to

$$(1 - X/2) \sum_{v \in V_j^k} b_v > (1 - X/2) \sum_{v \in (V^k - V_j^k)} b_v.$$

If $X \geq 2$, then we get $0 > 0$ or $\sum_{v \in V_j^k} b_v < \sum_{v \in (V^k - V_j^k)} b_v$, both of which are contradictions (note that the latter one violates the condition of the second case). Therefore, $X \leq 1$ ($X = 0$ or $X = 1$).

Now that V_j^k contains at most one element, Γ_j^k can be calculated by

$$\left(\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v \right) - \left(\max_{\forall v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v \right),$$

leading to the traffic bandwidth in the second case.

Appendix C. Proof of Theorem 1

Assume we move a virtual node with guaranteed bandwidth Δ ($\Delta > 0$) from set $(V^k - V_j^k)$ to V_j^k . That is, placing a virtual node from VNR k on substrate node j . We draw a line over a symbol to denote the new value after the moving, e.g., $\overline{V_j^k}$ ($V_j^k \subset \overline{V_j^k}$) and $\overline{B_j^k}$. Here we prove $\overline{B_j^k} \geq B_j^k$. According to formula (1), we analyze Ψ_j^k and $\Lambda_j^k + \Gamma_j^k$.

Firstly, we compare $\overline{\Psi_j^k}$ with Ψ_j^k . For formula (2), since obviously $\sum_{v \in \overline{V_j^k}} b_v / 2 > \sum_{v \in V_j^k} b_v / 2$, we now investigate the second case. Specifically, we analyze three conditions as follows.

1. For the condition

$$\sum_{v \in (V^k - V_j^k)} b_v \leq 2 \max_{\forall v \in V_j^k} b_v - \sum_{v \in V_j^k} b_v$$

and

$$\sum_{v \in (V^k - \overline{V_j^k})} b_v \leq 2 \max_{\forall v \in \overline{V_j^k}} b_v - \sum_{v \in \overline{V_j^k}} b_v,$$

we can directly derive: $\max_{\forall v \in V_j^k} b_v \geq \sum_{v \in V^k} b_v / 2$, and $\max_{\forall v \in \overline{V_j^k}} b_v \geq \sum_{v \in V^k} b_v / 2$. Therefore,

$$\max_{\forall v \in V_j^k} b_v = \max_{\forall v \in \overline{V_j^k}} b_v.$$

Based on the second case of formula (2), we get

$$\begin{aligned} \overline{\Psi_j^k} &= \sum_{v \in \overline{V_j^k}} b_v - \max_{\forall v \in \overline{V_j^k}} b_v + \sum_{v \in (V^k - \overline{V_j^k})} b_v \\ &= \sum_{v \in V^k} b_v - \max_{\forall v \in \overline{V_j^k}} b_v. \end{aligned}$$

So $\overline{\Psi_j^k} = \Psi_j^k$.

2. For the condition

$$\sum_{v \in (V^k - V_j^k)} b_v > 2 \max_{\forall v \in V_j^k} b_v - \sum_{v \in V_j^k} b_v$$

and

$$\sum_{v \in (V^k - \overline{V_j^k})} b_v > 2 \max_{\forall v \in \overline{V_j^k}} b_v - \sum_{v \in \overline{V_j^k}} b_v,$$

we get

$$\overline{\Psi_j^k} = \max_{\forall v \in \overline{V_j^k}} b_v \geq \max_{\forall v \in V_j^k} b_v.$$

So $\overline{\Psi_j^k} \geq \Psi_j^k$.

3. For the condition

$$\sum_{v \in (V^k - V_j^k)} b_v > 2 \max_{\forall v \in V_j^k} b_v - \sum_{v \in V_j^k} b_v$$

and

$$\sum_{v \in (V^k - \overline{V_j^k})} b_v \leq 2 \max_{\forall v \in \overline{V_j^k}} b_v - \sum_{v \in \overline{V_j^k}} b_v,$$

we can directly derive: $\max_{\forall v \in V_j^k} b_v < \sum_{v \in V^k} b_v / 2$ and $\max_{\forall v \in \overline{V_j^k}} b_v \geq \sum_{v \in V^k} b_v / 2$. Based on the second case of formula (2), we get

$$\overline{\Psi_j^k} = \sum_{v \in \overline{V_j^k}} b_v - \max_{\forall v \in \overline{V_j^k}} b_v + \sum_{v \in (V^k - \overline{V_j^k})} b_v$$

$$\begin{aligned}
&= \sum_{v \in V_j^k} b_v + \sum_{v \in (V^k - \bar{V}_j^k)} b_v \\
&\geq \max_{v \in V_j^k} b_v.
\end{aligned}$$

$$\text{So } \bar{\Psi}_j^k \geq \Psi_j^k.$$

Next, we compare $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k$ with $\Lambda_j^k + \Gamma_j^k$, and turn to formulas (3) and (7). We also consider three conditions.

1. For the condition

$$\sum_{v \in V_j^k} b_v \geq \sum_{v \in (V^k - V_j^k)} b_v$$

and

$$\sum_{v \in \bar{V}_j^k} b_v \geq \sum_{v \in (V^k - \bar{V}_j^k)} b_v,$$

we can derive: $\Lambda_j^k + \Gamma_j^k = \bar{\Lambda}_j^k + \Delta$. Now we consider three sub-conditions. ① If both before and after the node moving, only the first condition of Eq. (7) is met, then $\bar{\Gamma}_j^k = \Gamma_j^k + \Delta$. ② If both before and after the node moving, only the second condition of (7) is met, then

$$\max_{v \in V_j^k} b_v - (\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v)/2$$

$$> \sum_{v \in (V^k - V_j^k)} b_v,$$

hence $\max_{v \in V_j^k} b_v > \sum_{v \in V^k} b_v/2$, so $\max_{v \in V_j^k} b_v > \Delta$. Therefore, $\bar{\Gamma}_j^k = \Gamma_j^k + \Delta$. ③ If before and after the node moving, the first and second conditions of (7) are met, respectively, then $\max_{v \in V_j^k} b_v = \Delta$. So

$$\begin{aligned}
\bar{\Gamma}_j^k - \Gamma_j^k &= \sum_{v \in V_j^k} b_v - (\sum_{v \in V_j^k} b_v - \sum_{v \in (V^k - V_j^k)} b_v)/2 \\
&= \sum_{v \in V^k} b_v/2.
\end{aligned}$$

Since we have assumed that $\sum_{v \in V_j^k} b_v > \sum_{v \in (V^k - V_j^k)} b_v$, we get $\Delta < \sum_{v \in V^k} b_v/2$. So $\bar{\Gamma}_j^k - \Gamma_j^k > \Delta$. To sum up the three sub-conditions, $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k \geq \Lambda_j^k + \Gamma_j^k$.

2. For the condition

$$\sum_{v \in V_j^k} b_v < \sum_{v \in (V^k - V_j^k)} b_v$$

and

$$\sum_{v \in \bar{V}_j^k} b_v < \sum_{v \in (V^k - \bar{V}_j^k)} b_v,$$

we can derive: $\bar{\Lambda}_j^k = \Lambda_j^k + \Delta$. According to Section 4.3, since no extra bandwidth remains, we get $\bar{\Gamma}_j^k = 0$ and $\bar{\Gamma}_j^k = 0$. So $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k > \Lambda_j^k + \Gamma_j^k$.

3. For the condition

$$\sum_{v \in V_j^k} b_v < \sum_{v \in (V^k - V_j^k)} b_v$$

and

$$\sum_{v \in \bar{V}_j^k} b_v \geq \sum_{v \in (V^k - \bar{V}_j^k)} b_v,$$

we can derive: $\Lambda_j^k = \sum_{v \in V_j^k} b_v$, and $\bar{\Gamma}_j^k = 0$, $\bar{\Lambda}_j^k = \sum_{v \in (V^k - \bar{V}_j^k)} b_v$.

If the first case of (7) is met, we get

$$\bar{\Lambda}_j^k + \bar{\Gamma}_j^k = \sum_{v \in V^k} b_v/2 > \sum_{v \in V_j^k} b_v,$$

so $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k = \Lambda_j^k + \Gamma_j^k$. If the second case of (7) is met, then $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k = \sum_{v \in V^k} b_v - \max_{v \in V_j^k} b_v$. Obviously, $\max_{v \in V_j^k} b_v \in \{\max_{v \in V_j^k} b_v, \Delta\}$. According to the condition of this branch, we get

$$\max_{v \in V_j^k} b_v \leq \sum_{v \in V_j^k} b_v < \sum_{v \in V^k} b_v/2,$$

so if $\max_{v \in V_j^k} b_v = \max_{v \in V_j^k} b_v$, we get

$$\bar{\Lambda}_j^k + \bar{\Gamma}_j^k > \sum_{v \in V^k} b_v/2 > \Lambda_j^k + \Gamma_j^k.$$

If $\max_{v \in V_j^k} b_v = \Delta$, then

$$\bar{\Lambda}_j^k + \bar{\Gamma}_j^k = \sum_{v \in V_j^k} b_v + \sum_{v \in (V^k - \bar{V}_j^k)} b_v \geq \Lambda_j^k + \Gamma_j^k.$$

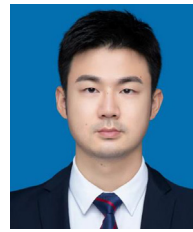
To sum up, $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k \geq \Lambda_j^k + \Gamma_j^k$.

Finally, according to the above analysis, we get $\bar{\Psi}_j^k \geq \Psi_j^k$ and $\bar{\Lambda}_j^k + \bar{\Gamma}_j^k \geq \Lambda_j^k + \Gamma_j^k$. According to formula (1), we get $\bar{B}_j^k \geq B_j^k$.

References

- [1] H. Yao, X. Chen, M. Li, P. Zhang, L. Wang, A novel reinforcement learning algorithm for virtual network embedding, *Neurocomputing* 284 (2018).
- [2] C.K. Dehury, P.K. Sahoo, DYVINE: Fitness-based dynamic virtual network embedding in cloud computing, *IEEE JSAC* 37 (5) (2019).
- [3] A. Song, W. Chen, Y. Gong, X. Luo, J. Zhang, A divide-and-conquer evolutionary algorithm for large-scale virtual network embedding, *IEEE Trans. Evol. Comput.* 24 (3) (2020).
- [4] Y. Yuan, Z. Tian, C. Wang, F. Zheng, Y. Lv, A Q-learning-based approach for virtual network embedding in data center, *Neural Comput. Appl.* 32 (7) (2020).
- [5] H.K. Thakkar, C.K. Dehury, P.K. Sahoo, MUVINE: Multi-stage virtual network embedding in cloud data centers using reinforcement learning-based predictions, *IEEE JSAC* 38 (6) (2020).
- [6] M. Alaluna, N. Neves, F.M.V. Ramos, Elastic network virtualization, in: *IEEE INFOCOM 2020*, 2020, pp. 814–823.
- [7] B. Beach, *Pro PowerShell for Amazon Web Service*, Springer, 2019 (Chapter Virtual Private Cloud).
- [8] Google cloud virtual private cloud, 2022, <https://cloud.google.com/vpc>. (Accessed 15 November 2022).
- [9] J. Duan, Z. Guo, Y. Yang, Cost efficient and performance guaranteed virtual network embedding in multicast fat-tree DCNs, in: *IEEE INFOCOM 2015*, 2015, pp. 136–144.
- [10] C.K. Dehury, P.K. Sahoo, Failure aware semi-centralized virtual network embedding in cloud computing fat-tree data center networks, *IEEE Trans. Cloud Comput.* (2020).
- [11] C. Chekuri, S. Khanna, On multi-dimensional packing problems, in: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1999, pp. 185–194.
- [12] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30, No. 1, 2016.
- [13] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, N. Freitas, Dueling network architectures for deep reinforcement learning, in: *ICML*, 2016.

- [14] L. Gong, Y. Wen, Z. Zhu, T. Lee, Toward profit-seeking virtual network embedding algorithm via global resource capacity, in: IEEE INFOCOM 2014, 2014, pp. 1–9.
- [15] S. Wang, J. Bi, J. Wu, A.V. Vasilakos, Q. Fan, VNE-TD: A virtual network embedding algorithm based on temporal-difference learning, *Comput. Netw.* 161 (2019).
- [16] Z. Li, F. Yuan, L. Ma, A load balancing algorithm for solving multi-objective virtual network embedding, *Trans. Emerg. Telecommun. Technol.* 33 (6) (2022).
- [17] Microsoft azure virtual network, 2022, <https://azure.microsoft.com/en-us/services/virtual-network/>. (Accessed 15 November 2022).
- [18] VPC routes in google cloud, 2022, <https://cloud.google.com/vpc/docs/routes>. (Accessed 07 December 2022).
- [19] Distributed virtual routing, 2022, <https://www.extremenetworks.com/resources/solution-brief/distributed-virtual-routing/>. (Accessed 07 December 2022).
- [20] Configure distributed virtual routing, 2022, https://access.redhat.com/documentation/en-us/red_hat_openshift_platform/8/html/networking_guide/sec-dvr. (Accessed 07 December 2022).
- [21] Distributed virtual routing - overview and east/west routing, 2022, <https://assafmuller.com/2015/04/15/distributed-virtual-routing-overview-and-eastwest-routing/>. (Accessed 07 December 2022).
- [22] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, M. Casado, The design and implementation of open vSwitch, in: *USENIX NSDI 2015*, 2015, pp. 117–130.
- [23] P. Emmerich, D. Raumer, F. Wohlfart, G. Carle, Performance characteristics of virtual switching, in: *Proceedings of 3rd IEEE International Conference on Cloud Networking, CloudNet*, 2014, pp. 120–125.
- [24] T. Begin, B. Baynat, G. Artero Gallardo, V. Jardin, An accurate and efficient modeling framework for the performance evaluation of DPDK-based virtual switches, *IEEE Trans. Netw. Serv. Manag.* 15 (4) (2018).
- [25] H. Cao, L. Yang, H. Zhu, Novel node-ranking approach and multiple topology attributes-based embedding algorithm for single-domain virtual network embedding, *IEEE Internet Things J.* 5 (1) (2018).
- [26] H. Cao, S. Wu, G.S. Aujla, Q. Wang, L. Yang, H. Zhu, Dynamic embedding and quality of service-driven adjustment for cloud networks, *IEEE Trans. Ind. Inform.* 16 (2) (2020).
- [27] C. Yu, Q. Lian, D. Zhang, C. Wu, PAME: Evolutionary membrane computing for virtual network embedding, *J. Parallel Distrib. Comput.* 111 (2018).
- [28] W. Fan, F. Xiao, X. Chen, L. Cui, S. Yu, Efficient virtual network embedding of cloud-based data center networks into optical networks, *IEEE Trans. Parallel Distrib. Syst.* 32 (11) (2021).
- [29] M. Ait aba, M. Elkael, B. Jouaber, H. Castel-Taleb, A. Araldo, D. Olivier, A two-stage algorithm for the virtual network embedding problem, in: *IEEE LCN 2021*, 2021, pp. 395–398.
- [30] M. Elkael, H. Castel-Taleb, B. Jouaber, A. Araldo, M.A. Aba, Improved Monte Carlo tree search for virtual network embedding, in: *IEEE LCN 2021*, 2021, pp. 605–612.
- [31] K.T. Nguyen, C. Huang, Distributed parallel genetic algorithm for online virtual network embedding, *Int. J. Commun. Syst.* 34 (4) (2021).
- [32] P. Zhang, C. Wang, N. Kumar, W. Zhang, L. Liu, Dynamic virtual network embedding algorithm based on graph convolution neural network and reinforcement learning, *IEEE Internet Things J.* 9 (12) (2022).
- [33] Z. Yan, J. Ge, Y. Wu, L. Li, T. Li, Automatic virtual network embedding: A deep reinforcement learning approach with graph convolutional networks, *IEEE JSAC* 38 (6) (2020).
- [34] A. Suzuki, S. Harada, Safe multi-agent deep reinforcement learning for dynamic virtual network allocation, in: *IEEE GLOBECOM 2020*, 2020, pp. 1–7.
- [35] H. Ballani, P. Costa, T. Karagiannis, A. Rowstron, Towards predictable datacenter networks, *ACM SIGCOMM Comput. Commun. Rev.* 41 (4) (2011).
- [36] J. Duan, Y. Yang, Efficient virtual network embedding for variable size virtual machines in fat-tree data centers, in: *Proceedings of the 45th International Conference on Parallel Processing, ICPP*, 2016, pp. 1–10.
- [37] L.T. Kou, G. Markowsky, Multidimensional bin packing algorithms, *IBM J. Res. Dev.* 21 (5) (1977).
- [38] D.A. Popescu, A.W. Moore, PTPmesh: Data center network latency measurements using PTP, in: *IEEE 25th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, MASCOTS*, 2017, pp. 73–79.
- [39] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 1998.
- [40] S.I. Abrita, M. Sarker, F. Abrar, M.A. Adnan, Benchmarking vm startup time in the cloud, in: *International Symposium on Benchmarking, Measuring and Optimization*, 2018, pp. 53–64.



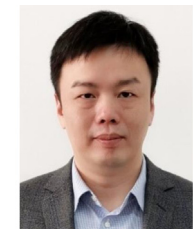
Meng Chen is currently a Research Assistant in the School of Big Data and Software Engineering, Chongqing University. He received the bachelor's degree from the School of Artificial Intelligence, Beijing Normal University, Beijing, China, in 2011, and the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2017. He worked as a senior cloud engineer in Huawei Cloud BU from Jul. 2017 to Nov. 2020. His research interests include virtual network embedding and mobile edge computing.



Jiaxin Hou is currently pursuing the master's degree with the College of Big Data and Software, Chongqing University, Chongqing, China. His current research interests include distributed data processing, spatio-temporal data mining, and deep reinforcement learning.



Yongpan Sheng is currently a Research Assistant at School of Big Data and Software Engineering, Chongqing University. He obtained his Ph.D. degree in Computer Architecture from the School of Computer Science and Engineering, University of Electronic Science and Technology of China. Prior to that, He received the bachelor degree in network engineering from the Chengdu University of Information Technology, and master degree in software engineering from University of Electronic Science and Technology of China in 2011 and 2014, respectively. His research interests include knowledge graphs and natural language processing.



Yingbo Wu received the Ph.D. degree from the School of Mechanical Engineering, Chongqing University, Chongqing, China, in 2012. From Jun. 2015 to Oct. 2015, he was a Visiting Scholar with the School of Engineering and Applied Science, University of Cincinnati USA. Since 2003, he has been with the School of Big Data and Software, Chongqing University, where he is currently a Full Professor and the Vice Director of the Department of Software Engineering. His research has primarily focused on the machine learning, intelligent optimization and decision.



Sen Wang is an Associate Professor in the School of Big Data and Software Engineering, Chongqing University, Chongqing, China. He received B.S., M.S., and Ph.D. degree in computer science in University of Science and Technology of China (USTC), Chinese Academy of Sciences (CAS) and Tsinghua University, China, in 2005, 2008 and 2014, respectively. His research interests include in-network caching, Information-centric Networking, Cloud Computing, Software Defined Networking and Network Functions Virtualization.



Jianyuan Lu received the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, in 2017. He got the B.S. degree from Information and Computing Science, Beijing University of Posts and Telecommunications, in 2011. He is currently a researcher in Alibaba Cloud. His research interests include cloud network measurements, cloud network anomaly detection, high performance network algorithm, and high performance cloud network forwarding.



Qilin Fan is currently a Lecturer in the School of Big Data and Software Engineering, Chongqing University, Chongqing, China. She received the B.E. degree in the College of Software Engineering, Sichuan University, Chengdu, China, in 2011, and the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2017. Her research interests include network optimization, mobile edge computing and caching, network virtualization and machine learning.